

TESI DI LAUREA IN
INFORMATICA

Utilizzo di telecamere in ambienti industriali reali

CANDIDATO

Michele Ongaro

RELATORE

Prof. Agostino Dovier

TUTOR AZIENDALE

Ing. Umberto Piconi

CONTATTI DELL'ISTITUTO

Dipartimento di Scienze Matematiche, Informatiche e Fisiche

Università degli Studi di Udine

Via delle Scienze, 206

33100 Udine — Italia

+39 0432 558400

<https://www.dmif.uniud.it/>

Indice

1	Introduzione	1
1.1	Obiettivi	1
1.2	Presentazione dello scenario applicativo	1
1.3	Considerazioni implementative	2
2	Sviluppo delle metriche	5
2.1	Metriche basate sull'analisi delle frequenze	6
2.1.1	Fourier	7
2.1.2	Skewness	9
2.2	Metriche basate sull'edge detection	11
2.2.1	Laplacian	11
2.2.2	Canny	11
2.3	Altre metriche	12
3	Definizione dei threshold	15
3.1	Calcolo del threshold	15
3.2	Considerazioni sulla risoluzione delle immagini	16
4	Valutazione e confronto delle metriche	19
5	Conclusioni e considerazioni finali	21

1

Introduzione

1.1 Obiettivi

L'obiettivo del presente lavoro è sviluppare un algoritmo in grado di determinare automaticamente lo stato di pulizia dell'obiettivo di una telecamera, identificando eventuali condizioni di sporco o ostruzione attraverso l'analisi delle immagini acquisite. Le immagini considerate provengono da un flusso video generato da un insieme di telecamere IP installate in prossimità di specifiche targhe (*plate*). Ciascuna targa riporta un codice visivo progettato secondo una codifica di Hamming, che ne garantisce la riconoscibilità e la robustezza rispetto a errori di lettura. L'idea alla base dell'algoritmo è sfruttare la presenza di tali targhe come riferimento noto all'interno della scena. Analizzando la qualità con cui il codice viene acquisito e decodificato dalle immagini, è possibile ricavare indicatori utili a valutare le condizioni dell'obiettivo. Fenomeni quali accumulo di polvere, sporco, gocce d'acqua o altre ostruzioni tendono infatti a degradare la qualità dell'immagine, riducendo la nitidezza e aumentando gli errori nella rilevazione del codice. Il sistema dovrà quindi elaborare i frame del flusso video e produrre una classificazione dello stato della telecamera, distinguendo tra condizioni operative normali e situazioni in cui l'obiettivo risulta sufficientemente sporco da compromettere l'affidabilità dell'acquisizione e della successiva lettura del codice.

1.2 Presentazione dello scenario applicativo

L'infrastruttura aziendale considerata è costituita da un insieme di telecamere IP installate in posizioni strategiche all'interno dell'impianto produttivo. Tali telecamere sono orientate verso delle cisterne contenenti materiale metallico fuso e hanno il compito di monitorarne il movimento e l'identificazione. Ogni cisterna è dotata di una targa (*plate*) sulla quale è riportato un codice opportunamente progettato per essere riconosciuto automaticamente da un sistema di visione artificiale. Le telecamere trasmettono continuamente le immagini acquisite mediante protocollo RTSP (*Real Time Streaming Protocol*) ad una pipeline di elaborazione dedicata al riconoscimento dei codici presenti sui *plate*. La pipeline è composta da diversi livelli di elaborazione che, partendo dall'immagine acquisita, eseguono una serie di operazioni finalizzate all'individuazione del codice e alla sua successiva decodifica. Al termine del processo viene restituito il valore numerico associato alla targa osservata. Non sempre, tuttavia, la pipeline riesce

a completare correttamente il processo di riconoscimento. Le cause di un fallimento possono essere molteplici e non necessariamente legate a malfunzionamenti del sistema di visione. Una prima causa è rappresentata dall'assenza del *plate* all'interno della porzione di immagine analizzata. Infatti, per motivi di efficienza e configurabilità, il sistema di riconoscimento può essere impostato per elaborare esclusivamente una specifica regione dell'immagine originale (*Region of Interest*, ROI). Di conseguenza, se il *plate* si trova al di fuori di tale area oppure non è visibile nell'inquadratura in quel determinato istante, la pipeline non dispone delle informazioni necessarie per effettuare la decodifica. Una seconda categoria di problematiche riguarda invece la qualità dell'immagine acquisita. Condizioni di illuminazione non adeguate, errori di messa a fuoco, movimenti della cisterna, presenza parziale del *plate* nell'inquadratura o una dimensione insufficiente dello stesso all'interno dell'immagine possono compromettere significativamente le prestazioni dell'algoritmo di riconoscimento. In tali situazioni il codice risulta difficilmente distinguibile e il livello di affidabilità richiesto dalla decodifica non può essere garantito. Particolare interesse riveste un'ulteriore causa di degrado delle prestazioni, ovvero la presenza di sporco sull'obiettivo della telecamera. Durante le normali operazioni dell'impianto può accadere che il materiale metallico fuso generi schizzi che raggiungono la superficie dell'obiettivo, depositandosi su di essa. La presenza di tali impurità altera la qualità delle immagini prodotte, introducendo sfocature, zone opache e riduzioni del contrasto che rendono più difficile il riconoscimento del codice.

Poiché il sistema di decodifica opera imponendo una soglia minima di affidabilità, il degrado della qualità visiva può portare la pipeline a rifiutare il risultato della lettura, producendo un esito negativo anche quando il *plate* è effettivamente presente e correttamente inquadrato. In questi casi il mancato riconoscimento non dipende dall'assenza del codice, ma esclusivamente dalle condizioni non ottimali del sistema di acquisizione. Per questo motivo risulta particolarmente utile disporre di un indicatore indipendente dalla presenza o meno del *plate* all'interno dell'immagine, capace di fornire una valutazione oggettiva dello stato dell'obiettivo della telecamera. Un tale indicatore consentirebbe di individuare tempestivamente situazioni di degrado della qualità delle immagini e di programmare interventi di manutenzione prima che il problema influisca significativamente sul processo produttivo. L'obiettivo finale è quindi quello di realizzare un sistema in grado di monitorare automaticamente le condizioni dell'obiettivo e segnalare eventuali anomalie dovute alla presenza di sporco. In questo modo sarebbe possibile ridurre i periodi di inattività della pipeline di riconoscimento, migliorando l'affidabilità complessiva del sistema e garantendo una maggiore continuità operativa.

1.3 Considerazioni implementative

Le telecamere impiegate nell'infrastruttura sono configurate per gestire automaticamente la messa a fuoco dell'immagine. Tale funzionalità consente normalmente di mantenere nitida la scena osservata anche in presenza di variazioni della distanza tra la telecamera e gli oggetti inquadrati.

Dalle osservazioni effettuate sul campo è emerso tuttavia un comportamento ricorrente quando l'obiettivo viene colpito da schizzi di materiale metallico fuso. In queste situazioni la telecamera tende a considerare la macchia depositata sulla superficie dell'obiettivo come nuovo punto di riferimento per il sistema di autofocus. Di conseguenza, la messa a fuoco viene regolata sulla macchia stessa anziché sugli elementi presenti nella scena osservata. Questo fenomeno produce un evidente degrado della qua-

lità dell'immagine: la porzione di sporco risulta relativamente nitida mentre il resto della scena appare fortemente sfocato. In tali condizioni la pipeline di riconoscimento dei codici non è più in grado di garantire il livello di affidabilità richiesto, poiché le informazioni necessarie alla corretta individuazione e decodifica del *plate* risultano degradate.

Alla luce di queste osservazioni, lo studio presentato in questo lavoro si concentra esclusivamente sull'analisi del grado di nitidezza delle immagini acquisite. L'ipotesi di partenza è che la perdita di fuoco rappresenti il principale effetto osservabile della presenza di sporco sull'obiettivo e che, di conseguenza, possa essere utilizzata come indicatore indiretto dello stato della telecamera. L'obiettivo dell'algoritmo non è quindi quello di rilevare direttamente la presenza di macchie o impurità sull'obiettivo, bensì quello di distinguere tra immagini nitide e immagini sfocate. Una classificazione affidabile di tali condizioni consentirebbe infatti di individuare automaticamente le situazioni in cui la telecamera non opera più nelle condizioni ottimali previste. In particolare, qualora l'algoritmo determini che un'immagine risulta significativamente sfocata, tale informazione potrà essere interpretata come un forte indicatore di una perdita di corretta messa a fuoco. Nel contesto applicativo considerato, e sulla base delle evidenze raccolte, questa situazione è associata con elevata probabilità alla presenza di sporco sull'obiettivo causato dagli schizzi di metallo provenienti dalle cisterne monitorate.

2

Sviluppo delle metriche

La determinazione del grado di nitidezza di un'immagine rappresenta un problema ampiamente studiato nell'ambito dell'*image processing* e della visione artificiale. Nel corso degli anni sono state proposte numerose tecniche per stimare la qualità visiva di un'immagine e, più in particolare, per individuare la presenza di fenomeni di sfocatura che possono compromettere le prestazioni dei sistemi di riconoscimento automatico. Nel contesto di questo lavoro, l'obiettivo è individuare un insieme di metriche in grado di fornire una misura quantitativa della nitidezza delle immagini acquisite dalle telecamere. Tali metriche dovranno successivamente essere utilizzate per discriminare tra immagini che presentano condizioni di acquisizione corrette e immagini che risultano degradate a causa di una perdita di messa a fuoco. Le tecniche più diffuse per affrontare questo problema possono essere ricondotte a due grandi categorie. La prima comprende le **metriche basate sull'analisi delle frequenze**, mentre la seconda raccoglie le **metriche basate sull'individuazione dei bordi** (*edge detection*).

Le metriche appartenenti alla prima categoria sfruttano il fatto che un'immagine nitida contiene generalmente una maggiore quantità di dettagli ad alta frequenza. Quando l'immagine diventa sfocata, tali dettagli tendono a scomparire e il contenuto in alta frequenza si riduce. Analizzando quindi la distribuzione delle frequenze presenti nell'immagine è possibile ottenere una stima del suo livello di nitidezza.

Le metriche basate sull'*edge detection*, invece, si fondano sull'osservazione che un'immagine ben focalizzata presenta contorni e transizioni di intensità particolarmente marcati. Al contrario, la sfocatura tende ad attenuare tali variazioni rendendo i bordi meno definiti. Misurando la quantità dei bordi rilevati è quindi possibile ricavare informazioni utili sul grado di messa a fuoco dell'immagine.

Sebbene queste due categorie rappresentino gli approcci maggiormente utilizzati in letteratura, non esauriscono tutte le possibili tecniche disponibili. Esistono infatti ulteriori metriche che analizzano altre proprietà statistiche o strutturali dell'immagine e che possono fornire informazioni complementari sulla sua qualità visiva. Nel corso di questo capitolo verranno quindi esaminate diverse metodologie, evidenziandone i principi di funzionamento, i vantaggi e i limiti nel contesto applicativo considerato.

Prima di applicare una qualsiasi delle metriche descritte, le immagini vengono sottoposte ad una fase preliminare di *preprocessing*. Lo scopo di questa fase è ridurre l'influenza di fattori esterni che potrebbero alterare il risultato delle misurazioni e rendere meno confrontabili immagini acquisite in condizioni differenti. In particolare, il processo di *preprocessing* adottato prevede due operazioni principali. La

prima consiste nella normalizzazione della luminosità dell'immagine, necessaria per limitare gli effetti dovuti a variazioni delle condizioni di illuminazione della scena. La seconda consiste nella conversione dell'immagine dalla rappresentazione a colori alla scala di grigi. Questa trasformazione consente di ridurre la complessità computazionale dell'elaborazione e di concentrare l'analisi esclusivamente sulle variazioni di intensità luminosa, che costituiscono l'informazione maggiormente rilevante per la stima della nitidezza. Una volta completata questa fase preliminare, l'immagine quindi risulta pronta per l'applicazione delle diverse metriche che verranno descritte nelle sezioni successive.

2.1 Metriche basate sull'analisi delle frequenze

Uno degli approcci più utilizzati per valutare il grado di nitidezza di un'immagine consiste nell'analizzarne il contenuto in frequenza. Questo metodo si basa sull'idea che un'immagine possa essere interpretata come un segnale bidimensionale discreto, nel quale a ciascuna coppia di coordinate spaziali è associato un valore di intensità luminosa. In maniera analoga a quanto avviene nell'elaborazione dei segnali monodimensionali, anche per le immagini è possibile studiare come l'informazione sia distribuita tra diverse componenti di frequenza. In questo contesto, il concetto di frequenza non è legato al tempo, bensì alla variazione spaziale dell'intensità dei pixel. Più precisamente, una variazione lenta e graduale dei livelli di grigio corrisponde a una componente a bassa frequenza, mentre variazioni rapide e localizzate corrispondono a componenti ad alta frequenza. Dal punto di vista visivo, le componenti a bassa frequenza descrivono principalmente la struttura generale dell'immagine, come le forme di grandi dimensioni e le variazioni graduali di luminosità. Le componenti ad alta frequenza, invece, rappresentano i dettagli più fini della scena, come i contorni degli oggetti, le texture e le transizioni nette tra regioni adiacenti. La sfocatura agisce proprio su queste componenti ad alta frequenza. Quando un'immagine perde nitidezza, i dettagli più fini tendono infatti ad attenuarsi o a scomparire completamente. Di conseguenza, il contenuto in alta frequenza si riduce mentre le componenti a bassa frequenza diventano predominanti. Questo comportamento rende possibile stimare il grado di messa a fuoco di un'immagine osservando come l'energia del segnale sia distribuita nello spettro delle frequenze. Per effettuare tale analisi è possibile utilizzare la Trasformata Discreta di Fourier (*Discrete Fourier Transform*, DFT), uno strumento matematico che consente di passare dal dominio spaziale al dominio delle frequenze. Applicando la DFT ad un'immagine si ottiene una rappresentazione alternativa del contenuto informativo, nella quale è possibile osservare direttamente le diverse componenti di frequenza che contribuiscono alla formazione dell'immagine originale.

L'ipotesi alla base delle metriche presentate in questa sezione è quindi la seguente: un'immagine correttamente focalizzata possiede generalmente una maggiore quantità di componenti ad alta frequenza rispetto ad un'immagine sfocata. Analizzando opportunamente lo spettro ottenuto tramite la trasformata di Fourier è quindi possibile costruire un indice capace di discriminare tra immagini nitide e immagini degradate. Le metriche descritte nelle sezioni successive condividono entrambe questo principio di funzionamento e si basano sull'applicazione della Trasformata Discreta di Fourier. Esse differiscono tuttavia nel modo in cui lo spettro delle frequenze viene analizzato e nella particolare misura utilizzata per quantificare il contenuto informativo associato alle alte frequenze.

2.1.1 Fourier

La prima metrica considerata si basa sull'analisi dello spettro delle frequenze ottenuto tramite la Trasformata Discreta di Fourier (DFT) [6]. L'idea alla base di questo approccio è quella di valutare come l'energia dell'immagine sia distribuita tra le componenti a bassa e ad alta frequenza.

Come discusso nella sezione precedente, le immagini nitide tendono a contenere una quantità maggiore di componenti ad alta frequenza, associate alla presenza di dettagli, contorni e variazioni rapide di intensità. Al contrario, la sfocatura agisce come un filtro passa-basso, attenuando progressivamente tali componenti e concentrando gran parte dell'energia dell'immagine nelle frequenze più basse. Per effettuare questa analisi si applica innanzitutto la Trasformata Discreta di Fourier all'immagine pre-processata. Il risultato è una matrice di numeri complessi che rappresenta lo spettro delle frequenze dell'immagine. Ciascun elemento della matrice descrive il contributo di una specifica frequenza alla formazione dell'immagine originale. Dopo aver eseguito l'operazione di *shift* della trasformata, la componente a frequenza nulla (detta anche componente continua o *DC*) viene posizionata al centro della matrice. Procedendo radialmente verso l'esterno si incontrano frequenze via via più elevate. In altre parole, il centro della matrice rappresenta le basse frequenze, mentre le regioni periferiche contengono le componenti ad alta frequenza. Poiché i valori restituiti dalla trasformata sono numeri complessi, per ogni elemento viene calcolato il modulo. In questo modo si ottiene una matrice contenente esclusivamente valori reali positivi, comunemente indicata come *magnitude spectrum*. Tali valori rappresentano l'intensità con cui ciascuna frequenza contribuisce all'immagine. Una volta ottenuta la matrice dei moduli, si procede alla separazione delle frequenze basse da quelle alte. A tale scopo viene definita una regione circolare centrata nello spettro e avente raggio pari al 10% della dimensione minima tra altezza e larghezza dell'immagine. Tutte le frequenze contenute all'interno di questo cerchio vengono considerate basse frequenze, mentre tutte quelle esterne vengono considerate alte frequenze.

Indicando con Ω l'insieme di tutte le componenti della trasformata di Fourier e con Ω_{low} il sottoinsieme delle basse frequenze, viene definito il seguente indice:

$$\alpha = \frac{\sum_{f \in \Omega_{\text{low}}} \|f\|}{\sum_{f \in \Omega} \|f\|} \quad (2.1)$$

dove:

- α rappresenta il valore di sfocatura (*blur value*);
- Ω è l'insieme di tutte le componenti della trasformata di Fourier;
- Ω_{low} contiene esclusivamente le componenti associate alle basse frequenze;
- $\|f\|$ indica il modulo della componente complessa f .

L'indice α assume valori compresi tra 0 e 1 e può essere interpretato come la frazione di energia concentrata nelle basse frequenze. Quando l'immagine è nitida, una parte significativa dell'energia è distribuita anche nelle frequenze più elevate. In questo caso il denominatore risulta molto maggiore del numeratore e il valore di α tende ad assumere valori relativamente bassi. Al contrario, quando l'immagine è sfocata, il contenuto ad alta frequenza diminuisce sensibilmente e la maggior parte dell'energia si

concentra nelle basse frequenze. Di conseguenza il numeratore e il denominatore assumono valori sempre più simili e l'indice α tende ad avvicinarsi a 1. Ne consegue che valori bassi di α sono generalmente associati ad immagini nitide, mentre valori elevati indicano una probabile perdita di messa a fuoco.

Lo pseudocodice riportato di seguito descrive l'implementazione della metrica.

```

1 def getBlurValue(image):
2     omega = fft(image)           # trasformata di fourier
3     magnitude = abs(omega)       # calcolo moduli
4     h, w = shape(magnitude)
5     cutoff = min(h, w) * 0.1    # limite per le basse frequenze
6
7     low = magnitude[where r < cutoff].sum()
8     high = magnitude[where r >= cutoff].sum()
9
10    return low / (high + low)

```

Le Figure successive mostrano la rappresentazione della magnitude della trasformata di Fourier per due immagini caratterizzate da un diverso livello di nitidezza. Nel caso dell'immagine sfocata, gran parte dell'energia risulta concentrata nella regione centrale dello spettro. Questo comportamento indica una predominanza delle basse frequenze e una ridotta presenza di dettagli ad alta frequenza. Al contrario, nell'immagine nitida si osserva una distribuzione più ampia dell'energia anche nelle regioni periferiche della trasformata. La presenza di contributi significativi alle alte frequenze è infatti direttamente collegata alla presenza di dettagli e contorni ben definiti nella scena originale.

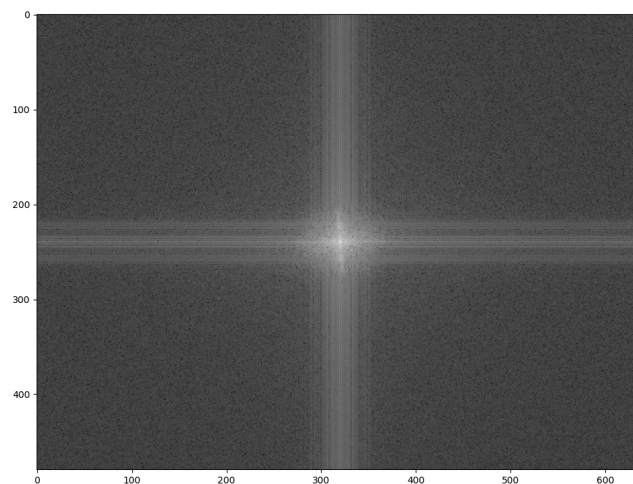


Figura 2.1: Magnitude della trasformata di Fourier per un'immagine sfocata. L'energia risulta prevalentemente concentrata nelle basse frequenze.

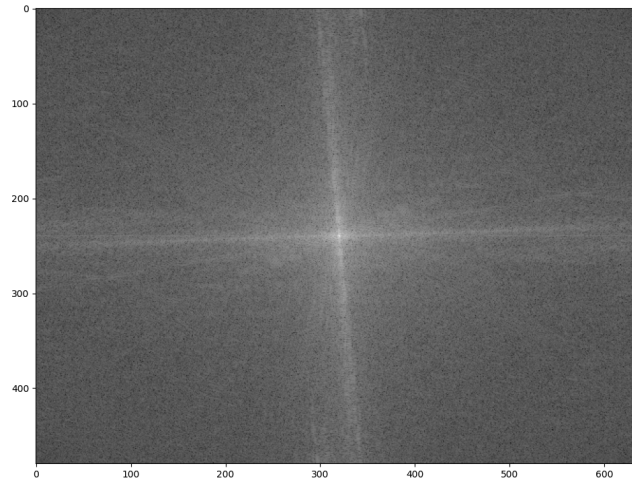


Figura 2.2: Magnitude della trasformata di Fourier per un'immagine nitida. Si osserva una maggiore presenza di componenti ad alta frequenza rispetto al caso precedente.

2.1.2 Skewness

Questa metrica fa uso dell'indice di *skewness* [2], ossia un valore che permette di misurare la forma della funzione di distribuzione delle frequenze. Per costruire la curva di distribuzione vanno considerati i valori assunti dai moduli degli elementi della trasformata di Fourier andando a campionare i valori in cerchi concentrici centrati nella frequenza 0. L'indice di *skewness* ci dirà poi se la distribuzione è *right-skewed* con un numero positivo o *left-skewed* con un numero negativo. In particolare ci aspettiamo una distribuzione *right-skewed* siccome i contributi delle basse frequenze contribuiscono in maniera più significativa, specialmente quando l'immagine diventa più sfocata. L'indice di *skewness* sarà poi ottenuto tramite una formula, come ad esempio quella di *Pearson*:

$$\text{Pearson's median skewness} = 3 \cdot \frac{\text{Mean} - \text{Median}}{\text{Standard deviation}} \quad (2.2)$$

Come si può constatare dalle immagini che seguono, l'idea è quella che un'immagine sfocata ha una curva *right-skewed* più accentuata rispetto a quella di una immagine più nitida, quindi questa metrica darà un valore più alto che sta a significare che la curva si sta muovendo sempre di più da una distribuzione normale.

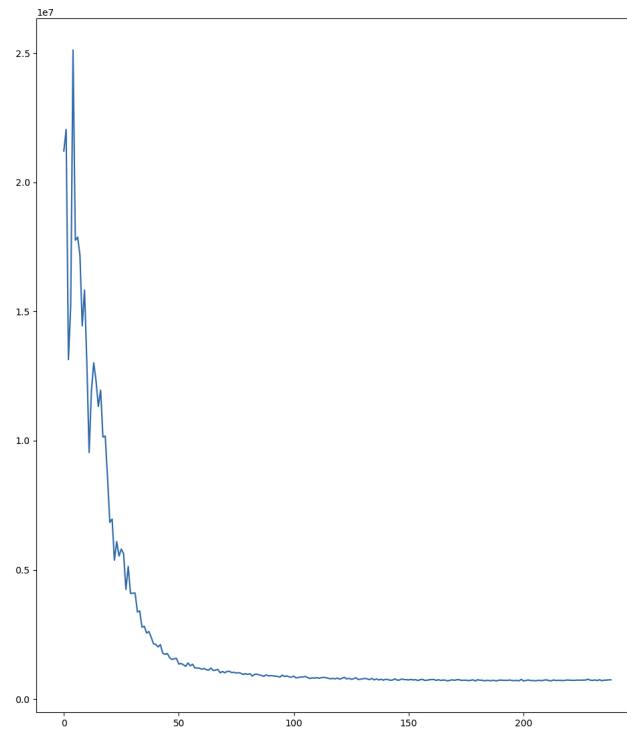


Figura 2.3: Somma magnitudini per frequenza di una immagine sfocata

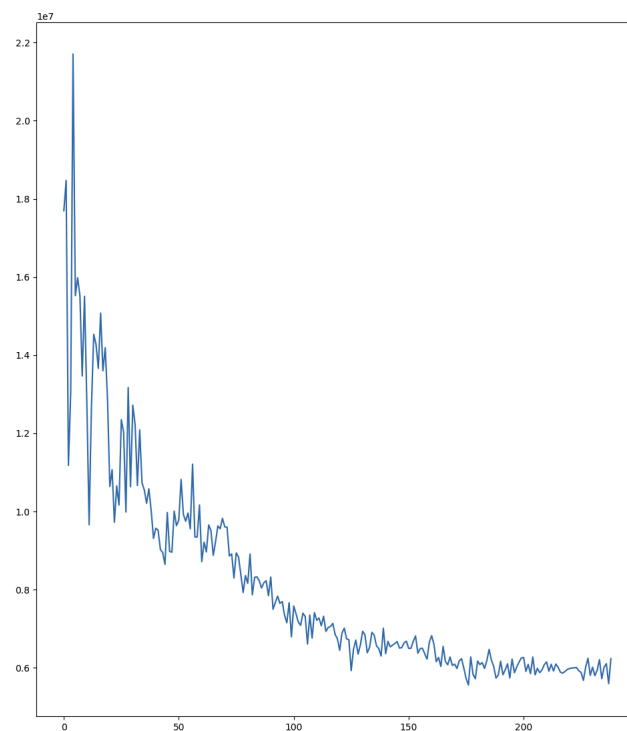


Figura 2.4: Somma magnitudini per frequenza di una immagine nitida

Dopo aver calcolato l'indice di *skewness* per normalizzare il risultato e mantenerlo all'interno dell'intervallo $[0, 1]$ è possibile applicare la funzione sigmoide. Lo pseudocodice che descrive il calcolo per questa metrica è quindi il seguente:

```

1 def getBlurValue(image):
2     omega = fft(image)
3     magnitude = abs(omega)
4     h, w = shape(fft)
5
6     radiusValues = []
7     for (radius = 1, radius < min(w,h), radius++):
8         radiusValues.push(magnitude[r == radius].sum())
9
10    mean = mean(radiusValues)
11    median = median(radiusValues)
12    std = std(radiusValues)
13    if std == 0:
14        return 1
15    skew = 3 * (mean - median) / std
16    return sigmoid(skew)

```

2.2 Metriche basate sull'edge detection

Per quanto riguarda le metriche basate sull'*edge detection* il concetto alla base delle diverse interpretazioni è che un'immagine nitida ha dei bordi ben definiti e in quantità maggiore rispetto ad un'immagine sfocata.

2.2.1 Laplacian

Questa metrica prevede l'utilizzo del filtro Laplacian [3]. Questo filtro permette di calcolare la derivata spaziale di una immagine e lo fa rilevando le regioni dell'immagine che subiscono dei cambiamenti di intensità immediati. Una volta ottenuta la derivata, valutandone la sua varianza sarà possibile creare un indice inversamente proporzionale al livello di sfocatura: se la varianza è elevata allora l'immagine sarà nitida e questo deriva dal fatto tale tipo di immagini possiede solitamente dei cambiamenti piuttosto rapidi dei valori assunti dai vari pixel.

```

1 def getBlurValue(image):
2     edges = laplacian(image)
3     if edges.var() == 0:
4         return 1
5     return 1 / edges.var()

```

2.2.2 Canny

In maniera completamente analoga alla metrica Laplacian appena vista, con l'operatore Canny [4] è possibile ricavare una particolare immagine di output che mostra le discontinuità di intensità rilevate. Canny si suddivide in più fasi. Inizialmente all'immagine di partenza viene applicata una convoluzione Gaussiana per rimuovere il rumore, ossia le frequenze più alte, poi tramite un filtro di derivata spaziale come Sobel [5] o analoghi esegue il calcolo della derivata prima. Gli spigoli quindi danno origine alle creste presenti nell'immagine gradiente, queste creste vengono prese in considerazione dall'algoritmo che imposta a 0 i pixel che non hanno come valore di cresta massimale. La metrica che poi va a rilevare

la sfocatura dell'immagine considera la varianza dell'immagine risultante, questo valore è direttamente proporzionale alla nitidezza ma inversamente proporzionale alla sfocatura perciò se la varianza è nulla il valore di sfocatura (*blurValue*) sarà 0, il che vuol dire che l'immagine è completamente sfocata, se però la varianza è strettamente maggiore allora la metrica ne calcolerà il reciproco in modo che la metrica possa crescere insieme al livello di sfocatura dell'immagine

```
1 def getBlurValue(image):
2     edges = canny(image)
3     if edges.var() == 0:
4         return 1
5     return 1 / edges.var()
```

2.3 Altre metriche

Le metriche basate sull'analisi delle frequenze e le metriche basate sull'edge detection non sono le sole, infatti è possibile creare un indice per stabilire se l'immagine è sfocata o meno anche osservando la dimensione in byte dell'immagine JPEG [1]. Le immagini JPEG sfocate infatti occupano poco spazio perché la compressione JPEG è basata sulla perdita di dettaglio ad alta frequenza, che sono esattamente quelli che definiscono la nitidezza e i bordi definiti. Per una immagine sfocata il processo di compressione ha meno informazioni complesse da eliminare o ridurre, risultando in file più piccoli rispetto ad una immagine completamente nitida. Questa soluzione ha il vantaggio di essere molto semplice ma bisogna tenere a mente che è una metrica molto delicata siccome due immagini nitide con stessa risoluzione potrebbero avere delle dimensioni completamente diverse, questo perché la dimensione dipende da altri fattori come ad esempio il range di colori usato nell'immagine, o la dimensione che i dettagli assumono all'interno dell'immagine. Nel grafico seguente viene riportato l'andamento della dimensione di una immagine JPEG 640x480 che viene via via sfocata in maniera sempre più accentuata attraverso un kernel di dimensione sempre maggiore.

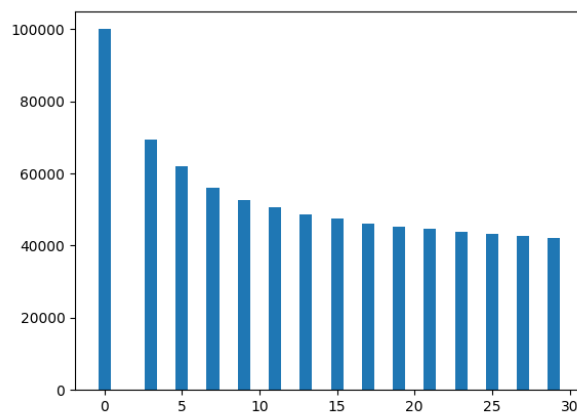


Figura 2.5: Dimensione di una immagine JPEG che viene via via sfocata

Un'altro sistema che si può utilizzare per costruire una metrica che permetta di capire se una immagine è sfocata o no consiste nell'analisi della varianza dell'immagine stessa. L'immagine infatti non è

altro che un vettore bidimensionale contenente dei valori numerici che vanno a definire il colore di ogni pixel e la varianza è direttamente proporzionale alla nitidezza dell'immagine. Un'immagine monocromatica, ossia l'immagine sfocata per antonomasia, avrà una varianza nulla e via via che aggiungiamo dettagli all'immagine otterremo una varianza sempre maggiore.

3

Definizione dei threshold

3.1 Calcolo del threshold

Le metriche che sono state definite offrono un indice numerico che può venire usato per determinare il livello di *blur* di una immagine, ma per stabilire se una immagine sia sfocata o meno bisogna stabilire un limite (*threshold*) al di sopra del quale si può dire con buona approssimazione che l'immagine sia sfocata, e idealmente al di sotto del quale l'immagine è ancora nitida.

Ogni metrica avrà una soglia diversa siccome si basano su approcci, come è stato mostrato, diversi ma sicuramente sarà un valore t compreso nell'intervallo $[0, 1]$. Avremo quindi che $t_{fourier}$ sarà diverso da t_{skew} che a sua volta sarà diverso da $t_{laplacian}$ e così via. In pratica ogni *threshold* t dovrà venire considerato in relazione ad una certa metrica. Supponendo di aver scelto una certa metrica il valore t potrà essere determinato tramite i valori $t_1, t_2, \dots, t_N$ dove N è il numero di telecamere presenti nell'infrastruttura aziendale. Ogni telecamera quindi contribuirà al risultato finale offrendo un *threshold* locale t_i che a sua volta sarà determinato dallo studio delle immagini acquisite da tale telecamera. Supponiamo che l'infrastruttura aziendale offra un numero pari a 3 telecamere. Avremo che $N = 3$ e quindi avremo a disposizione i valori t_1, t_2, t_3 che sono rispettivamente i *threshold* da utilizzare per capire se una immagine è sfocata rispettivamente per la telecamera 1, 2 e 3. Il risultato generale t potrà poi essere ottenuto dalla media:

$$t = \frac{t_1 + t_2 + t_3}{3} \quad (3.1)$$

Più in generale per un numero N di telecamere avremo che:

$$t = \frac{1}{N} \sum_{i=1}^N t_i \quad (3.2)$$

Supponiamo che ogni telecamera i metta a disposizione un certo numero di immagini n_i . Consideriamo la telecamera 10 per esempio, il numero di immagini messe a disposizione da tale ripresa sarà n_{10} . Ogni telecamera metterà quindi a disposizione un certo numero di n_i di *funzioni immagine*. Una *funzione immagine* accetta come parametro la dimensione del kernel da usare per sfocare l'immagine e restituisce l'immagine sfocata con il kernel scelto. Quindi $I_i^j(1)$ rappresenta la j -esima immagine (con $1 \leq j \leq n_i$) della telecamera i (con $1 \leq i \leq N$) sfocata con un kernel di dimensione 1, ossia la j -esima

immagine ripresa dalla telecamera i nella sua condizione originale. Se volessimo ottenere la 3^a immagine della telecamere 10 sfocata con un kernel di dimensione 29 calcoleremo: $I_{10}^3(29)$. Si ricorda che la dimensione del kernel va fornita tramite un numero dispari siccome la matrice kernel che applicherà la sfocatura tramite un filtro gaussiano, viene applicata tramite una operazione di convoluzione, in altre parole è necessario avere un valore centrale della matrice siccome questo verrà centrato sul pixel su cui verrà applicata l'operazione. La definizione di questa particolare funzione è importante siccome servirà a formalizzare un modo per ottenere i diversi t_i .

A questo punto per ogni telecamera bisognerà considerare le diverse immagini, ogni immagine infatti fornirà un *threshold* indicizzato da t_i^j dove i è il numero della telecamera $1 \leq i \leq N$ e j è il numero dell'immagine $1 \leq j \leq n_i$. Per ottenere t_i^j bisogna calcolare il risultato di $I_i^j(1), I_i^j(3), I_i^j(5) \dots$ e per ognuna di queste immagini via via sempre più sfocate verificare se il riconoscitore riesce o meno a identificare il numero codificato dal *plate*. A partire da una certa dimensione del kernel si osserverà che il riconoscitore fallirà in continuazione, sia k_i^j la dimensione del kernel per cui da $k_i^j + 1$ in poi il riconoscitore darà sempre esito negativo i.e. le immagini $I_i^j(k_i^j + 1), I_i^j(k_i^j + 2), \dots$ non saranno nitide a sufficienza per poter riconoscere il codice del *plate*. Per ottenere t_i^j sarà sufficiente calcolare il *blur value* dell'immagine $I_i^j(k_i^j + 1)$:

$$t_i^j = \alpha_{I_i^j(k_i^j+1)} \quad (3.3)$$

dove α_ξ è il *blur value* calcolato con la metrica scelta per l'immagine ξ .

Successivamente per ottenere il valore di t_i bisognerà combinare i vari t_i^j , una media può andare bene ma altre funzioni come ad esempio la mediana si possono usare se si reputa che nel *dataset* ci siano dei casi particolari che altererebbero in maniera significativa il risultato.

$$t_i = \frac{1}{n_i} \sum_{j=1}^{n_i} t_i^j \quad (3.4)$$

In questo modo possiamo andare a definire dei t diversi per ogni metrica, che possiamo chiamare $t^{fourier}$, t^{skew} , $t^{laplacian}$ e t^{canny} e saranno definiti complessivamente dalla formula generica:

$$t^m = \frac{1}{N} \sum_{i=1}^N \left(\frac{1}{n_i} \sum_{j=1}^{n_i} \alpha_{I_i^j(k_i^j+1)}^m \right) \quad (3.5)$$

dove m è la metrica di scelta e α_ξ^m rappresenta il *blur value* dell'immagine ξ calcolata tramite la metrica m .

3.2 Considerazioni sulla risoluzione delle immagini

È importante tenere a mente che telecamere diverse possono essere messe in posizioni diverse rispetto ai *plate* e dato che la risoluzione dell'immagine è fissa può succedere che per certe telecamere il *plate* risulterà occupare più pixel rispetto ad altre, questo può essere causato da un posizionamento alternativo della telecamera. Questa caratteristica è piuttosto rilevante nella definizione di un *threshold* siccome renderà l'immagine più o meno sensibile alla sfocatura. Una angolazione che riprende un *plate* da molto

vicino renderà la targa molto grande, facendole occupare più pixel rispetto ad una altra angolazione dove il *plate* viene posizionato più distanza, e questo comporta una diminuzione della sensibilità al blur: è possibile sfocare l'immagine con un kernel molto più grande e ottenere comunque un alto livello di affidabilità da parte del riconoscitore di codici. Questo è un problema siccome altera il valore del *threshold*, in questo caso particolare andando ad alzarlo. Se il limite viene considerato per-telecamera allora il problema non si pone più siccome il limite t sarà semplicemente un indice che rappresenta il limite di sfocatura che tale telecamere può supportare prima che l'immagine diventi troppo sfocata, ma se si desidera ottenere un t generico che vada bene per tutte le telecamere e che indichi se l'immagine è fuori fuoco o meno allora bisognerà prendere dei provvedimenti in modo da attenuare i contributi dati dalle telecamere che riprendono i *plate* troppo da vicino e accentuare quelli delle telecamere che hanno un *plate* molto distante. La *pipeline* di riconoscimento del plate prima di eseguire l'algoritmo che va a calcolare il codice si occupa di eseguire una preprocessing dell'immagine in modo da ridimensionarla in maniera tale che un lato del plate abbia una dimensione fissa: circa 15px. Esistono quindi dei coefficienti messi a disposizione dell'infrastruttura che permettono di normalizzare la dimensione dei plate per le varie telecamere. Considerare le immagini ridimensionate comporta dei vantaggi in termini di sensibilità alla sfocatura siccome i kernel usati dal filtro di sfocatura hanno una dimensione fissa 3,5,7,... Se quindi le immagini utilizzate nella formula non sono quelle direttamente ottenute dalla telecamere ma la loro versione ridimensionata allora sarà possibile ottenere dei valori di threshold $t_1, t_2, \dots, t_N$ più omogenei che permettono poi di definire un valore t che indica con più precisione il punto in cui l'immagine diventa sfocata.

Valutazione e confronto delle metriche

Ogni metrica porterà alla definizione di un certo t^m . Tale risultato deriva dal calcolo dei diversi *threshold* t_i ottenuti dalle diverse telecamere. Per capire quindi se una certa metrica sia buona si può osservare la relazione che è presente tra i vari t_i . L'idea è che se i vari *threshold* derivanti dalle diverse telecamere portano a un valore univoco allora la metrica starà offrendo un limite chiaro che permette di stabilire la nitidezza di un'immagine. Questo è il motivo per cui il valore t^m viene calcolato facendo la media di vari t_i . Per stabilire quindi se i vari *threshold* forniti dalle varie telecamere hanno dei valori analoghi si può osservare la loro deviazione standard. Ogni telecamere quindi offrirà un certo insieme di valori $t_i \in [0, 1]$. Il valore medio sarà $\mu = t^m$ con una deviazione standard σ .

Al fine di ottenere un confronto valido tra le diverse metriche sarà necessario normalizzare la deviazione standard calcolata fatto 1 il valor medio.

$$\sigma_{\text{norm}} = \frac{\sigma}{\mu} \quad (4.1)$$

In questo modo si può fare un confronto tra le diverse metriche e vedere quale sia quella con una deviazione standard più bassa, ossia la metrica che riesce a identificare meglio un limite che vada meglio per tutte le varie telecamere. I risultati ottenuti empiricamente per le diverse metriche sono le seguenti:

Metrica	μ	σ	σ_{norm}
Laplacian	$1.2684 \cdot 10^{-1}$	$9.8223 \cdot 10^{-2}$	$7.7 \cdot 10^{-1}$
Canny	$1.0884 \cdot 10^{-3}$	$1.4378 \cdot 10^{-3}$	1.3
Jpgsize	$2.6709 \cdot 10^{-2}$	$1.1889 \cdot 10^{-2}$	$4.5 \cdot 10^{-1}$
Fourier	$6.2948 \cdot 10^{-1}$	$7.0088 \cdot 10^{-1}$	$1.1 \cdot 10^{-1}$
Skew	1.3705	$2.3715 \cdot 10^{-1}$	$1.7 \cdot 10^{-1}$

(4.2)

Si noti che la metrica con la deviazione standard più bassa è quella che utilizza la "metrica di Fourier". È interessante notare che le metriche basate sullo studio delle frequenze in generale si sono comportate meglio rispetto a quelle basate sull'edge detection, tali metriche infatti risultano avere una varianza inferiore circa i valori di *threshold* assegnati dalle diverse telecamere. L'indice di skewness ha una varianza maggiore e questo è dovuto al fatto che anche se si basa fortemente sulla matrice calcolata

dalla trasformata di Fourier l'indice finale è determinato dalla forma che la curva dei magnitudi di frequenze assume. Questo indice può quindi essere leggermente sviante siccome non dipende dai valori propriamente assunti dai vari contributi ma dalla forma assunta dalla curva.

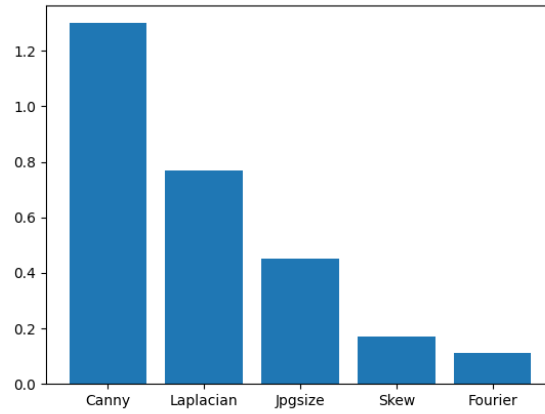


Figura 4.1: Un confronto delle deviazioni standard normalizzate dei vari t_i per le varie metriche

5

Conclusioni e considerazioni finali

Arrivati a questo punto è facile generare un algoritmo che permetta di classificare delle immagini non presenti nel dataset come sfocate o meno. Sia J^i una immagine acquisita dalla telecamera i , per capire se tale immagine sia stata scattata da un obiettivo sporco o meno è sufficiente eseguire un confronto tra il suo *blur value* α e quello ottenuto t .

$$\text{dirtTest}(J^i) = \begin{cases} \text{True} & t^m < \alpha_{J^i}^m \\ \text{False} & \text{else} \end{cases} \quad (5.1)$$

Se si desidera avere un confronto più mirato è possibile gestire il test in maniera locale per ogni telecamera usando il valore di *threshold* ottenuto dalla specifica telecamera:

$$\text{dirtTest}(J^i) = \begin{cases} \text{True} & t_i < \alpha_{J^i}^m \\ \text{False} & \text{else} \end{cases} \quad (5.2)$$

È stato mostrato come implementare e configurare un algoritmo che permetta di determinare se una immagine e più in generale un video sia stato prodotto da una telecamera con un obiettivo sporco. Sono state illustrate diverse tecniche che si possono utilizzare per il rilevamento di un certo valore chiamato *blur value* utilizzato per determinare se una certa immagine sia sfocata o meno se confrontato con un certo valore di *threshold* indicato da t unico a ogni metrica. Tale informazione può venire usata per eseguire della manutenzione preventiva sulle telecamere con obiettivo sporco in modo tale da rendere efficiente e produttivo il sistema di rilevamento codici dei *plate* posizionati sulle varie cisterne.

Bibliografia

- [1] CCITT. *T.81 – DIGITAL COMPRESSION AND CODING OF CONTINUOUS-TONE STILL IMAGES – REQUIREMENTS AND GUIDELINES*. International Telecommunication Union, 1992.
- [2] N. Balakrishnan NL. Johnson, S. Kotz. *Continuous Univariate Distributions. Vol. 1 (2 ed.)*. Wiley, 1994.
- [3] L. Shapiro R. Haralick. *Computer and Robot Vision pp 346-351*. Addison-Wesley, 1992.
- [4] Zhang Wei Sun Lining Rong Weibin, Li Zhanjing. *An improved Canny edge detection algorithm, pp 577-582*. IEEE, 2014.
- [5] Irwin Sobel. *History and Definition of the Sobel Operator*. 2014.
- [6] Gilbert Strang. *Wavelets*. American Scientist, Volume 82, Issue 3, 1994.